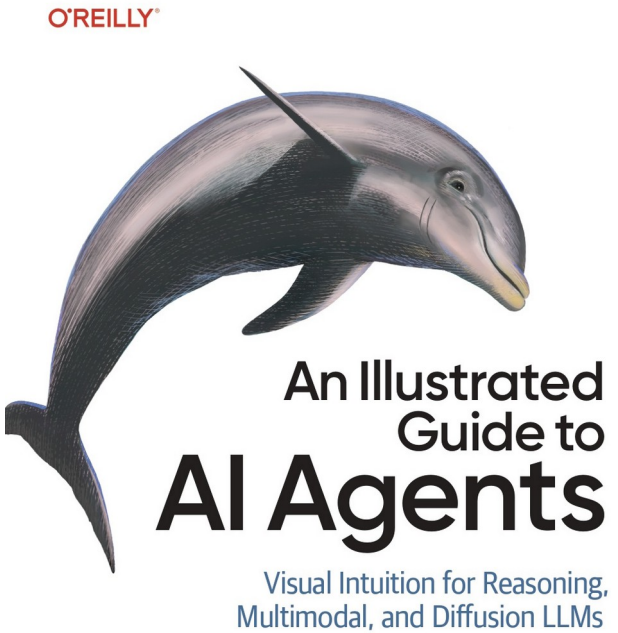
M2

LLMs & Reasoning: The Brain

Module 2 — Giving the TinyAgent a brain



307307 — BI Methods for Generative AI

Agentic AI · University of Petra

Lecturer: _____ Date: _____

Learning Objectives

1. Explain what an LLM does: tokens, next-token prediction, and the autoregressive loop.
2. Describe how an LLM is trained: pre-training, then post-training (SFT + RL).
3. Use the chat message format — system, user, assistant.
4. Explain temperature and the context window, and why they matter for agents.
5. Explain what a reasoning LLM is (System 1 vs System 2) and when to use one.

Where We Are — Adding the Brain

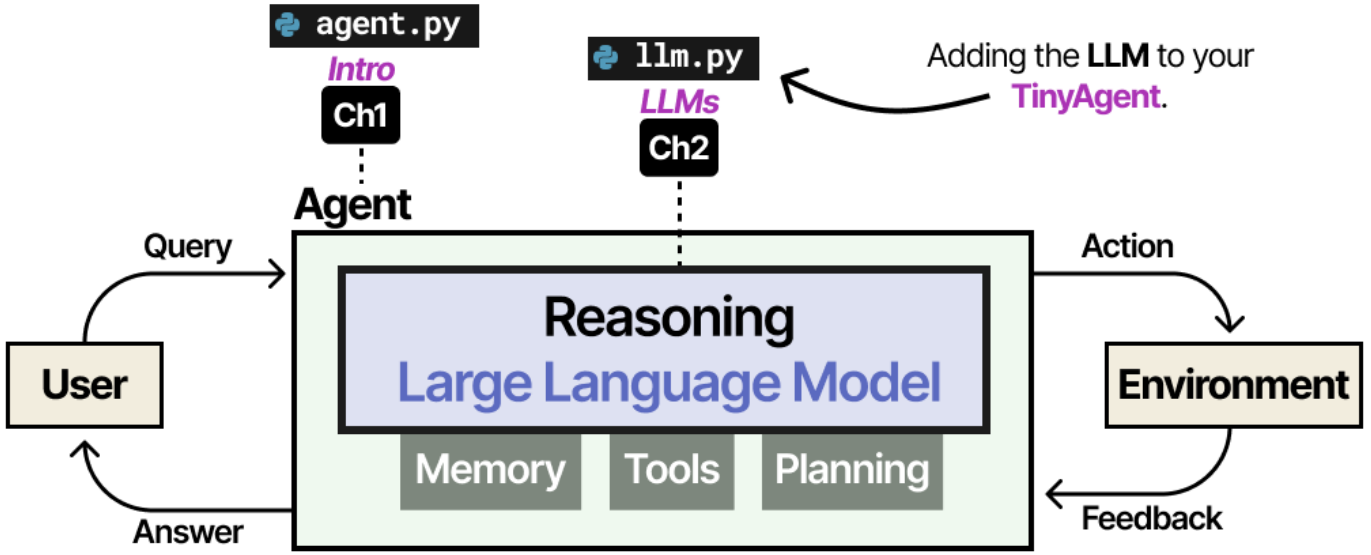


Figure from An Illustrated Guide to AI Agents (O'Reilly, 2026)

Module 1

We built the skeleton — it only echoed you.

Module 2 (today)

Add the LLM: a real, reusable brain (llm.py).

Next

Memory, tools, planning build on this brain.

What Is an LLM?

An LLM predicts the next token.

A token is a small piece of text — a word, part of a word, a number, or punctuation.
"flamingos" becomes two tokens: "flamingo" + "s".

1. **Split —** text is broken into tokens
2. **Predict —** the model scores the most likely next token
3. **Repeat —** build the answer token by token

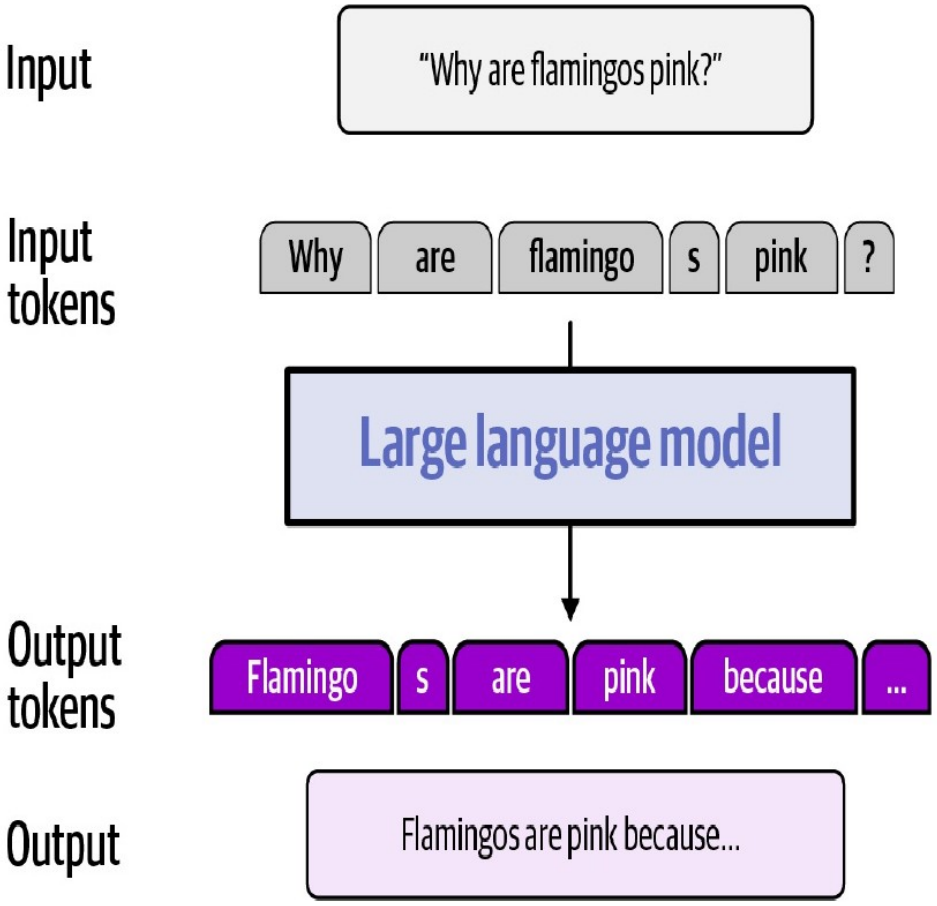


Figure 2-3, An Illustrated Guide to AI Agents (O'Reilly, 2026)

Autoregression — One Token at a Time

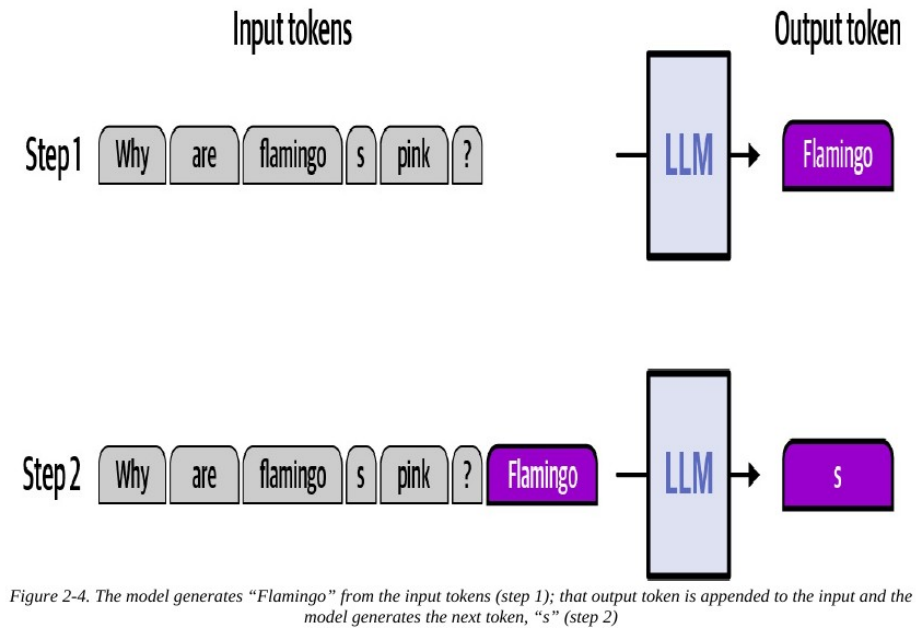


Figure 2-4. The model generates "Flamingo" from the input tokens (step 1); that output token is appended to the input and the model generates the next token, "s" (step 2)

Figure 2-4, An Illustrated Guide to AI Agents (O'Reilly, 2026)

The model generates a token, appends it to the input, and predicts again. Because it consumes its own output as new input, this loop is called autoregression — it is how a full answer is built, one token at a time.

How Does It Learn?

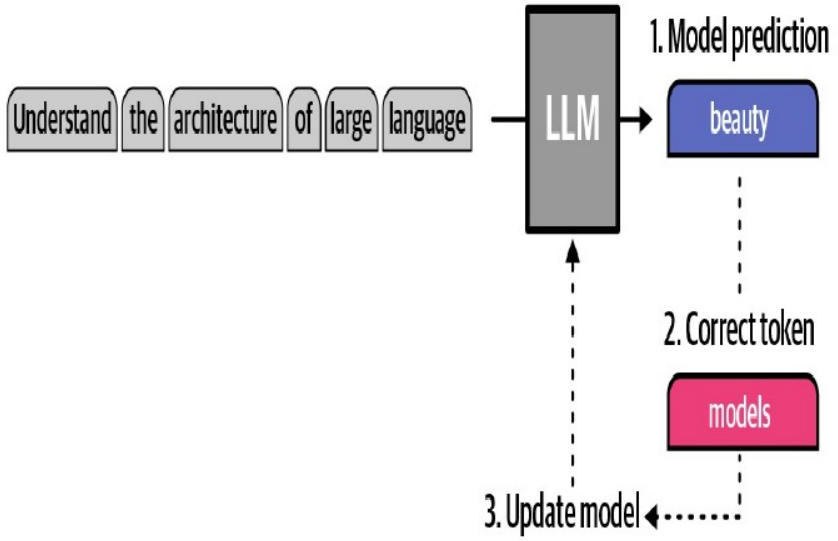
Pre-training

Read huge amounts of text and play one game billions of times: hide the next token, guess it, check, adjust. Result: a base model with broad knowledge.

Post-training

SFT (supervised fine-tuning): show example prompt → good answer pairs so it follows instructions.

RL (reinforcement learning): score the answers and nudge the model toward higher-scoring ones (RLHF, RLVR).



Pre-training as next-token prediction — Figure 2-8 (O'Reilly, 2026)

Talking to an LLM — The Chat Format

We send a list of messages, each with a role. The model replies as the assistant.

system	Privileged instruction that sets behavior before the user speaks. “You are a helpful study planner.”
user	What the person asks.
assistant	What the model replies.

```
messages = [

  {'role': 'system',
   'content': 'You are a tutor.'},

  {'role': 'user',
   'content': 'What is a token?'},

]

resp = client.chat.completions
          .create(model, messages)
```

The response also carries token counts and (later) reasoning and tool-call fields.

Temperature & Decoding

The model scores every possible next token. The decoding strategy picks one.

Temperature 0 always takes the highest-probability token — focused and repeatable.

Higher temperature allows sampling lower-probability tokens — more varied, less predictable.

For agents that must behave reliably, we usually keep temperature low.

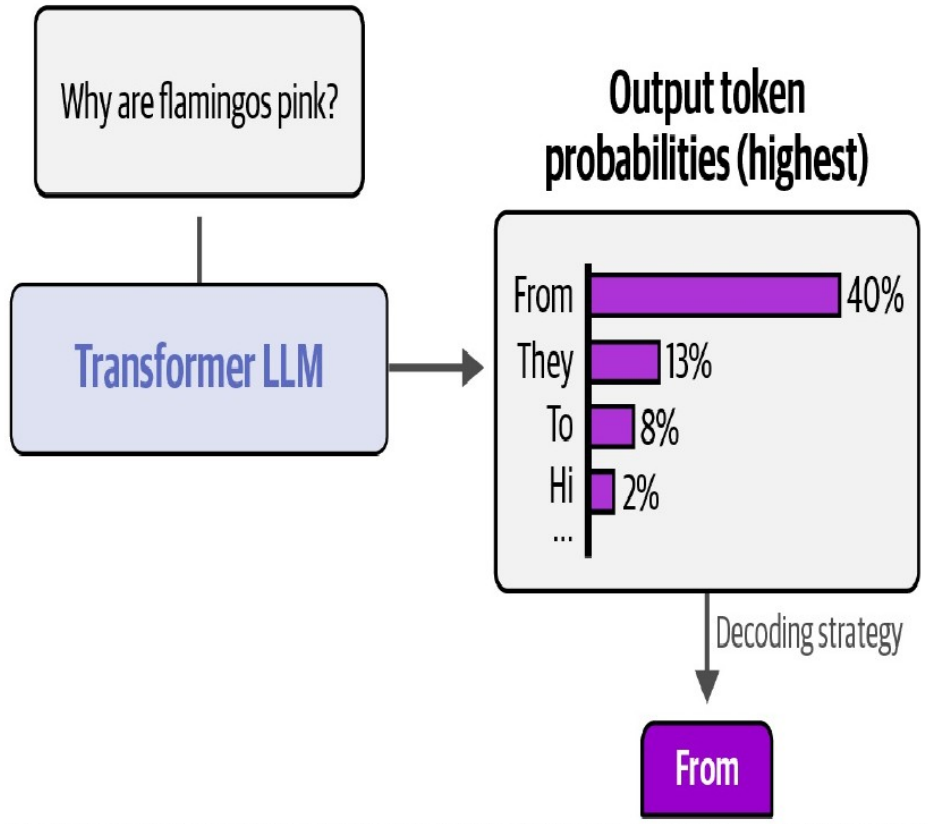


Figure 2-17. The decoding strategy selects a token from the probability distribution, where higher temperatures increase the chance of lower-probability tokens

Figure 2-17, An Illustrated Guide to AI Agents (O'Reilly, 2026)

The Context Window

The context window (context length) is the maximum number of tokens the model can handle at once — input plus output.

1

Everything must fit

All the agent 'knows' this turn — instructions, history, tool results — shares one budget.

2

More is not always better

Too much context can overwhelm the model and hurt its answers (information overload).

3

Context engineering

Give the model the right information, not all of it. This is the Memory module's job, next.

Reasoning LLMs — Think Before Answering

A reasoning LLM generates reasoning tokens — a chain of thought — before the final answer.

System 1 (fast, intuitive) = a non-reasoning LLM: quick answers from learned patterns.

System 2 (slow, deliberate) = a reasoning LLM: works through the problem step by step and catches mistakes.

Kahneman's dual-process theory (System 1 / System 2).

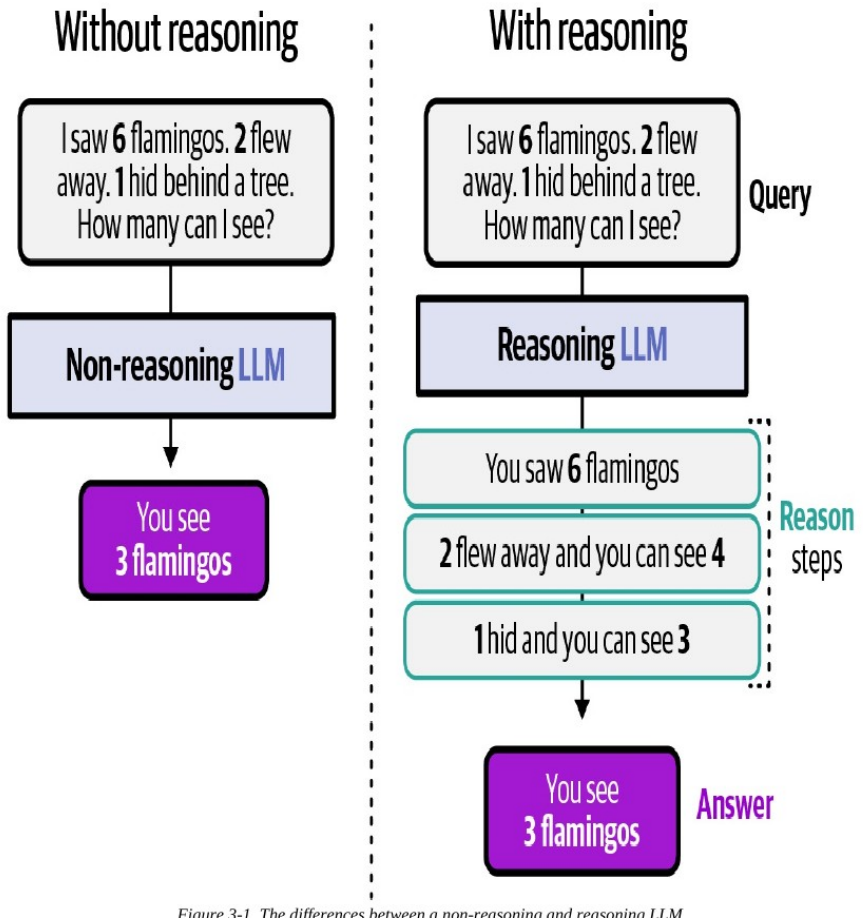


Figure 3-1, An Illustrated Guide to AI Agents (O'Reilly, 2026)

Thinking Longer — and When to Use It

The shift: test-time compute

Instead of only making models bigger (train-time), let them think longer at answer time. More 'thinking' tokens on hard problems tends to give better answers.

Why agents need it

Planning, choosing tools, and reflecting on results all require step-by-step thought. Reasoning is what makes those possible.

Rule of thumb

Reasoning model — complex, multi-step problems where accuracy matters.

Fast (non-reasoning) model — simple, high-volume tasks where speed and cost matter.

Choosing & Connecting to a Model

Open vs proprietary

Open-weight models (Gemma, Qwen) you can run yourself; proprietary models (Gemini, GPT, Claude) you call as a service.

Local vs cloud

Run locally with Ollama / LM Studio, or use a cloud free tier like Google Gemini or OpenRouter.

One endpoint to rule them

Most expose an OpenAI-compatible endpoint: same request shape, so one wrapper talks to any of them.

In this course: Google Gemini's free tier through the OpenAI-compatible endpoint — the same setup as your Practical folder.

What You Will Build Today

1 Chat format

Send system / user / assistant messages and read the reply.

2 LLM wrapper

A reusable brain (llm.py) over Gemini's OpenAI-compatible endpoint.

3 Temperature

See focused vs varied answers.

4 AgentLog + agent

Record each step and wire the brain into the TinyAgent.

Outcome: the TinyAgent goes from parroting to actually answering — but it is still stateless (no memory yet).

Key Takeaways

1. An LLM predicts the next token and builds answers autoregressively.
2. Training = pre-training (knowledge) + post-training (SFT + RL for helpfulness).
3. We talk to it with system / user / assistant messages; temperature and context window are key knobs.
4. A reasoning LLM thinks step-by-step (System 2) — essential for planning, tools, and reflection.
5. Models are interchangeable through the OpenAI-compatible endpoint; we use Gemini.

Discussion

Q1 What is a token, and what does 'autoregressive' mean?

Q2 What do the system, user, and assistant roles each do? What makes the system role special?

Q3 Give one task where you'd prefer a reasoning LLM and one where a fast model is better. Why?

Next: the Memory module — so the agent can hold a real conversation.